

Section 1 / Q2 — Single Source Shortest Path (SSSP) with MapReduce

Finds the shortest distance from node 0 to every node in a weighted directed graph, using iterative MapReduce (parallel Bellman-Ford). Written in Python, orchestrated with Bash.

Files

File	What it does
<code>prep.py</code>	Turns the raw edge list into the starting node state
<code>mapper.py</code>	One relaxation step: re-emits the graph + proposes new distances
<code>combiner.py</code>	Merges duplicate proposals locally, and partitions them across reducers
<code>reducer.py</code>	Picks the minimum distance per node, counts how many changed (one process per partition)
<code>format_output.py</code>	Prints the final node distance lines, INF if unreachable
<code>sssp_local.sh</code>	Runs the whole thing on one machine (<input> <output> <map tasks> <reducers>)
<code>sssp_slurm.sh</code>	Runs it on the cluster under Slurm (sbatch --ntasks=P)
<code>bench_slurm.sh</code>	Optional: benchmark sweep; REDUCERS=1 forces the single-reducer baseline
<code>gen_graph.py</code>	Makes random test graphs (same seed = same graph)
<code>dijkstra_ref.py</code>	Ordinary sequential Dijkstra, used to check our answers
<code>verify.sh</code>	Runs all tests and compares against Dijkstra
<code>tests/</code>	Small hand-made graphs (sample from the PDF, chain, cycle, etc.)

Input / output format

Input file:

```
V E      <- number of vertices, number of edges
u v w    <- one line per edge: from u, to v, weight w
...
```

Output:

```
0 0
1 3
2 2
3 7
```

<- "node_id distance", sorted by node, INF if unreachable

Running locally (your own computer)

You only need **python3** and **bash**. No Hadoop, no cluster.

Step 1 — go to this folder

```
cd "Section 1"
```

Step 2 — run on the sample from the assignment PDF

```
./sssp_local.sh tests/sample.txt output.txt
cat output.txt
```

You should see:

```
0 0
1 3
2 2
3 7
```

Step 3 — run on a bigger graph

PROF

First create one (**gen_graph.py V E seed**), then run on it:

```
python3 gen_graph.py 10000 50000 7 > big.txt
./sssp_local.sh big.txt output.txt 4 4
```

The last two numbers are how many map tasks and how many reducers to use. Both are optional (they default to 3 and 1). Any values work — the answer never depends on them, which is what **verify.sh** checks. To use your own graph file, put its name in place of **big.txt**.

With more than one reducer, the combiner partitions its output by **node_id % R** so that every record for a node reaches the same reducer, and the reducers then run concurrently.

Step 4 — check correctness against Dijkstra

```
./verify.sh
```

Prints **PASS/FAIL** for each test and ends with **ALL TESTS PASSED**.

Making a big graph for timing

```
python3 gen_graph.py 10000 50000 7 > big.txt      # V=10000, E=50000,
seed=7
./sssp_local.sh big.txt big_out.txt 4 4
python3 dijkstra_ref.py < big.txt > big_ref.txt
diff big_out.txt big_ref.txt && echo MATCH
```

If you get **Permission denied**, run **chmod +x *.sh *.py** once.

How it works (short version)

Each node's state is one line: **node \t dist|neighbour,weight;neighbour,weight;...**

Each iteration is one MapReduce job:

1. **Mapper** — for every node it re-emits the node's own line (so the graph survives to the next round) and, if the node is reachable, emits **neighbour \t D|dist+weight** for every out-edge. That is one Bellman-Ford relaxation.
2. **Local sort (sort)** — groups everything for the same node together within one mapper's output.
3. **Combiner** — collapses repeated proposals for the same node into one minimum, so less data crosses the network. With **R > 1** reducers it also partitions its output by **node_id % R**, which guarantees that every record for a node ends up in the same partition.
4. **Shuffle (sort -m)** — merges partition **r** across all mappers, so reducer **r** receives one sorted stream. The **R** merges are independent and run at the same time.
5. **Reducers** — each owns a disjoint set of nodes, so all **R** run concurrently. For each node one takes the minimum of its old distance and all proposals, writes the new line, and counts how many nodes improved. The new state is the concatenation of their outputs.
6. The driver repeats until the reducers' improvement counts **sum to 0** — at that point no edge can be relaxed further, so the distances are final.

Unreachable nodes never receive a proposal, stay at **1000000**, and are printed as **INF**. Converges in about the graph's hop-diameter rounds (22 rounds for

V=10000, E=50000), never more than V.

Correctness

`verify.sh` compares the MapReduce output byte-for-byte with `dijkstra_ref.py` on: the assignment sample, a chain, a cycle, an edgeless graph, a graph with an unreachable component, and 3 random 150-node graphs. Each case runs at several map-task and reducer counts, confirming the result depends on neither: mappers must not lose records at a chunk boundary, and the partitioner must not split one node's proposals across two reducers. It also checks that the iteration count is identical for every reducer count.